

Python for Data Analytics, Machine Learning & Generative AI

A long-term Python study reference for revision, interviews and projects

Learning path

1. Python Fundamentals
2. Data Analytics
3. Machine Learning
4. Generative AI
5. LLM Applications

Table of Contents

- 2. Python Fundamentals 4
 - 2.1 Variables 4
 - 2.2 Data Types 5
 - 2.2.1 int 5
 - 2.2.2 float 6
 - 2.2.3 str 8
 - 2.2.4 bool 9
 - 2.3 type() Function 10
 - 2.4 Type Conversion 11
 - 2.5 Operators 13
 - 2.5.1 Arithmetic Operators 13
 - 2.5.2 Comparison Operators 15
 - 2.5.3 Logical Operators 16
 - 2.5.4 Assignment Operators 17
 - 2.6 Practical Examples 18
 - 2.6.1 Student Marks and Average 19
 - 2.6.2 Sales Difference 19
 - 2.6.3 Sales Growth Percentage 20
 - 2.7 Section Summary 21

Planned Structure and Coverage Status

The sections below define the full structure of this reference, ordered from fundamentals to Data Analytics, Machine Learning, Generative AI and LLM applications. A section is written only after the topics in it have been learned. Sections marked Planned are listed to keep the structure stable and will be filled in as the material is covered.

No.	Section	Status	Covered so far
1	Python Overview	Planned	-
2	Python Fundamentals	In progress	Variables, int, float, str, bool, type(), type conversion, arithmetic, comparison, logical and assignment operators
3	Control Flow	Planned	-
4	Loops	Planned	-
5	Functions	Planned	-
6	Python Data Structures	Planned	-
7	Practical Python	Planned	-
8	NumPy for Data Analytics	Planned	-
9	Pandas for Data Analytics	Planned	-
10	Data Cleaning with Python	Planned	-
11	Exploratory Data Analysis	Planned	-
12	Data Visualization	Planned	-
13	Statistics with Python	Planned	-
14	SQL + Python	Planned	-
15	APIs + Python	Planned	-
16	Automation with Python	Planned	-
17	Python for Machine Learning	Planned	-
18	Python for Generative AI	Planned	-
19	Python for LLM Applications	Planned	-
20	Real-World Projects	Planned	-

2. Python Fundamentals

This section covers the building blocks of every Python program: storing values in variables, working with the core data types, converting between types, and combining values with operators. These ideas appear in everything that follows, from cleaning a column of data to computing evaluation metrics for a machine learning model or building a prompt for an LLM.

2.1 Variables

What it is

A variable is a name that refers to a value stored in memory. You create a variable by assigning a value to a name with the = operator. Python does not need a type declaration; the type comes from the value.

Why it matters

Variables let you store a value once and reuse it. In analytics code they hold totals, thresholds, file names and column names. In ML and GenAI code they hold settings such as learning rate, batch size, model name and prompt text.

Syntax

```
variable_name = value
```

Example

```
student_name = "Asha"  
marks = 87  
is_passed = True  
print(student_name, marks, is_passed)
```

Output

```
Asha 87 True
```

Explanation

- `student_name = "Asha"` stores a string and gives it the name `student_name`.
- `marks = 87` stores an integer.
- `is_passed = True` stores a boolean value.
- `print()` displays the values separated by a space.

Data/AI Example

```
dataset_name = "sales_2024.csv"  
row_count = 1250  
learning_rate = 0.01  
print(dataset_name, "has", row_count, "rows")  
print("Learning rate:", learning_rate)
```

Output

```
sales_2024.csv has 1250 rows  
Learning rate: 0.01
```

Important Points

- Variable names are case-sensitive: sales and Sales are different variables.
- Use meaningful snake_case names such as monthly_revenue instead of mr.
- The type belongs to the value, not to the name. A name can later refer to a value of another type, but doing this makes code harder to read.

Common Mistakes

- Using = when a comparison is needed. = assigns; == compares.
- Starting a name with a digit or using spaces or hyphens in it.
- Using a variable before assigning it, which raises NameError.
- Using built-in names such as list, str or sum as variable names, which hides the built-in.

Interview Notes

- **Q:** Is Python statically or dynamically typed?
A: Dynamically typed. The type is decided at run time from the value, and a name can later refer to a value of a different type.
- **Q:** What is the difference between = and ==?
A: = assigns a value to a name. == compares two values and returns a boolean.

Naming rules

Rule	Valid example	Invalid example
Start with a letter or underscore	total_sales	2nd_total
Use only letters, digits and underscores	unit_price	unit-price
Cannot be a Python keyword	class_name	class
Names are case-sensitive	sales and Sales differ	-

2.2 Data Types

A data type describes what kind of value a variable holds and which operations are allowed on it. Python has four basic types that appear in almost every data and AI program.

Type	Meaning	Example	Typical use
int	Whole numbers	42	Counts, quantities, IDs, epochs
float	Decimal numbers	3.14	Prices, ratios, accuracy scores
str	Text	"Python"	Names, categories, prompts
bool	True or False	True	Flags, conditions, filters

2.2.1 int

What it is

An int stores a whole number without a decimal point. It can be positive, negative or zero, and Python integers have no fixed size limit.

Why it matters

Integers represent counts and discrete quantities such as number of rows, students, orders, epochs or batch size.

Syntax

```
variable_name = 10
```

Example

```
students_present = 42
students_absent = 3
total_students = students_present + students_absent
print(total_students)
```

Output

```
45
```

Data/AI Example

```
epochs = 20
batch_size = 32
print("Training for", epochs, "epochs with batch size", batch_size)
```

Output

```
Training for 20 epochs with batch size 32
```

Important Points

- Integers are written without quotes and without a decimal point.
- Python integers can grow as large as memory allows, so there is no overflow.

Common Mistakes

- Writing `5.0` and expecting an integer. A value with a decimal point is a `float`.
- Putting quotes around a number. `"42"` is a string, not an integer.
- Writing leading zeros such as `007`, which is a syntax error in Python 3.

Interview Notes

- **Q:** What is the largest integer Python can store?
A: There is no fixed limit. It is limited only by available memory.

2.2.2 float

What it is

A `float` stores a number with a decimal point. Floats are used for measurements, prices and any value that is not a whole number.

Why it matters

Most real-world data is fractional: prices, temperatures, percentages, probabilities, model accuracy and learning rates.

Syntax

```
variable_name = 3.14
```

Examples

```
price = 500.0
discount_rate = 0.15
discount_amount = price * discount_rate
print(discount_amount)
```

Output

```
75.0
```

```
print(0.1 + 0.2)
print(round(0.1 + 0.2, 2))
```

Output

```
0.30000000000000004
0.3
```

Explanation

- Multiplying two floats gives a float.
- `round(value, 2)` rounds a number to 2 decimal places.
- The second example shows floating-point precision: `0.1 + 0.2` is not stored as exactly `0.3`.

Data/ML Example

```
learning_rate = 0.001
average_rating = 4.5
print("Learning rate:", learning_rate)
print("Average rating:", average_rating)
```

Output

```
Learning rate: 0.001
Average rating: 4.5
```

Important Points

- Floats are stored in binary, so many decimal values are only close approximations.
- Use `round()` when displaying results so tiny precision errors do not appear.

Common Mistakes

- Comparing floats with `==`. `0.1 + 0.2 == 0.3` is `False`. Round the values first before comparing.

Interview Notes

- **Q:** Why does `0.1 + 0.2` not equal `0.3` in Python?
A: Computers store floats in binary, and some decimal fractions cannot be represented exactly, so a very small error appears.

2.2.3 str

What it is

A str (string) stores text. A string is written inside single quotes, double quotes, or triple quotes for text that spans several lines.

Why it matters

Strings hold names, categories, column labels, file paths and, in GenAI work, the prompts sent to a language model.

Syntax

```
variable_name = "text"
variable_name = 'text'
variable_name = """multi-line text"""
```

Examples

```
product_name = "Wireless Mouse"
category = 'Accessories'
label = product_name + " - " + category
print(label)
print("-" * 20)
```

Output

```
Wireless Mouse - Accessories
-----
```

```
prompt = """You are a data analyst.
Summarize the sales trend."""
print(prompt)
```

Output

```
You are a data analyst.
Summarize the sales trend.
```

Explanation

- The + operator joins two strings. This is called concatenation.
- The * operator repeats a string, so "-" * 20 prints 20 dashes.
- Triple quotes keep line breaks, which is useful for long prompts.

Data/AI Example

```
instruction = "Summarize the following customer review."
review = "The delivery was fast and the product quality is excellent."
prompt = instruction + " " + review
print(prompt)
```

Output

```
Summarize the following customer review. The delivery was fast and the product quality is excellent.
```

Important Points

- Single and double quotes behave the same. Choose one style and stay consistent.
- `+` works only between two strings. Use `str()` to convert numbers first.
- `"10" + "20"` gives `"1020"`, not `30`.

Common Mistakes

- Adding a string and a number, for example `"Total: " + 500`, which raises `TypeError`. Write `"Total: " + str(500)` instead.
- Starting a string with one type of quote and ending it with another.

Interview Notes

- **Q:** What is the difference between `"5"` and `5`?
A: The first is a string (text). The second is an integer. They behave differently with operators.

2.2.4 bool

What it is

A `bool` (boolean) has only two possible values: `True` and `False`. Both are written with a capital first letter.

Why it matters

Booleans are the result of every comparison and the basis of conditions and data filters. In analytics they select rows that match a rule, such as sales above a target.

Syntax

```
variable_name = True
```

Example

```
marks = 72
is_passed = marks >= 40
print(is_passed)
```

Output

```
True
```

Explanation

- `marks >= 40` is a comparison that produces `True` or `False`.
- The result is stored in `is_passed`.

Data/BI Example

```
total_rows = 1000
missing_rows = 0
is_data_clean = missing_rows == 0
print(is_data_clean)
```

Output

```
True
```

Important Points

- Boolean names read well when they start with `is_` or `has_`.
- Internally `True` behaves like 1 and `False` like 0, so `True + True` is 2.

Common Mistakes

- Writing `true` or `false` in lowercase, which raises `NameError`.
- Writing `"False"` in quotes. That is a string, not a boolean.

Interview Notes

- **Q:** What does `True + True` return?
A: 2, because `bool` is a subtype of `int`.

2.3 type() Function

What it is

`type()` is a built-in function that returns the data type of a value or variable.

Why it matters

Checking types is the first step when debugging errors and when inspecting data read from files or APIs, where numbers often arrive as text.

Syntax

```
type(value)
```

Example

```
student_name = "Asha"
marks = 87
average = 87.5
is_passed = True
print(type(student_name))
print(type(marks))
print(type(average))
print(type(is_passed))
```

Output

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Data/AI Example

```
sensor_id = "SENSOR_01"  
temperature_reading = 28.6  
sensor_active = True  
print(type(sensor_id), type(temperature_reading), type(sensor_active))
```

Output

```
<class 'str'> <class 'float'> <class 'bool'>
```

Important Points

- `type()` only reports the type. It never changes it.
- A value in quotes is always a `str`, even if it looks like a number.

Common Mistakes

- Assuming data loaded from a file or typed by a user is numeric. It usually arrives as `str` and must be converted.

Interview Notes

- **Q:** What does `type(3.0)` return?
A: `<class 'float'>`.

2.4 Type Conversion

What it is

Type conversion changes a value from one data type to another. **Explicit conversion** is done by you with functions such as `int()`, `float()`, `str()` and `bool()`. **Implicit conversion** is done automatically by Python, for example when an integer is combined with a float.

Why it matters

Data from CSV files, APIs, forms and logs usually arrives as text. It must be converted to numbers before calculation, and numbers must be converted to text before they are joined into messages or prompts.

Syntax

```
int(value)  
float(value)  
str(value)  
bool(value)
```

Examples

```
quantity_text = "25"  
price_text = "199.50"  
quantity = int(quantity_text)  
price = float(price_text)  
total_cost = quantity * price  
print(total_cost)  
print(type(total_cost))
```

Output

```
4987.5  
<class 'float'>
```

```
print(int(3.9))  
print(float(7))  
print(str(100) + " units")  
print(bool(0), bool(1), bool(""), bool("Python"))
```

Output

```
3  
7.0  
100 units  
False True False True
```

```
quantity = 4  
unit_price = 49.5  
total_price = quantity * unit_price  
print(total_price)  
print(type(total_price))
```

Output

```
198.0  
<class 'float'>
```

Explanation

- Example 1 converts two text values to numbers before multiplying them.
- Example 2 shows that `int(3.9)` drops the decimal part. It truncates; it does not round.
- In example 3 Python converts `quantity` to a float automatically, so the result is a float.

Data/AI Example

```
accuracy_text = "92.5"  
accuracy = float(accuracy_text)  
error_rate = 100 - accuracy  
print(error_rate)
```

Output

```
7.5
```

Important Points

- `bool()` returns `False` for 0, 0.0 and an empty string, and `True` for other values.
- When an integer and a float are combined, the result is a float.

Common Mistakes

- `int("3.9")` raises `ValueError`. Convert with `int(float("3.9"))` instead.
- `int("abc")` also raises `ValueError` because the text is not a number.
- Expecting `int(3.9)` to give 4. It gives 3.

- Joining a string and a number without `str()`.

Interview Notes

- **Q:** What is the difference between implicit and explicit conversion?
A: Implicit conversion is done automatically by Python (`int + float` gives `float`). Explicit conversion is written by the programmer using `int()`, `float()`, `str()` or `bool()`.
- **Q:** What does `bool("False")` return?
A: `True`. Any non-empty string is true, even the text `"False"`.

2.5 Operators

An operator performs an operation on values (operands). Python groups operators by purpose. This section covers arithmetic, comparison, logical and assignment operators.

2.5.1 Arithmetic Operators

What it is

Arithmetic operators perform mathematical calculations on numbers.

Why it matters

Every analytics calculation uses them: totals, averages, differences, growth rates and scaled features for machine learning.

Syntax

```
result = operand_1 operator operand_2
```

Operator	Name	Example	Result
+	Addition	<code>10 + 3</code>	13
-	Subtraction	<code>10 - 3</code>	7
*	Multiplication	<code>10 * 3</code>	30
/	Division (always float)	<code>10 / 4</code>	2.5
//	Floor division	<code>10 // 3</code>	3
%	Modulus (remainder)	<code>10 % 3</code>	1
**	Exponent	<code>2 ** 3</code>	8

Examples

```
total_marks = 450
subjects = 5
print(total_marks + 50)
print(total_marks - 50)
print(total_marks * 2)
print(total_marks / subjects)
print(total_marks // 4)
print(total_marks % 4)
print(2 ** 3)
```

Output

```
500
400
900
90.0
112
2
8
```

```
print(10 + 2 * 3)
print((10 + 2) * 3)
```

Output

```
16
36
```

Explanation

- / always returns a float, so $450 / 5$ gives 90.0 .
- // keeps only the whole part of a division. % gives what is left over.
- Multiplication runs before addition, so $10 + 2 * 3$ is 16. Brackets change the order.

Data/BI Example

```
total_records = 1050
batch_size = 100
full_batches = total_records // batch_size
remaining_records = total_records % batch_size
print(full_batches, remaining_records)
```

Output

```
10 50
```

Important Points

- Order of evaluation: brackets, then **, then * / // %, then + -. Operators at the same level run from left to right.
- % is useful for checking even or odd numbers and for splitting data into groups.

Common Mistakes

- Expecting / to return an integer. Use // when a whole number is needed.
- Forgetting brackets, for example $a + b / 2$ instead of $(a + b) / 2$ for an average.
- Dividing by zero, which raises ZeroDivisionError.

Interview Notes

- **Q:** What is the difference between / and //?
A: / returns a float. // returns the floor of the division.
- **Q:** What does % do?
A: It returns the remainder of a division.

2.5.2 Comparison Operators

What it is

Comparison operators compare two values and return a boolean: True or False.

Why it matters

Comparisons are the foundation of conditions and filters, such as finding orders above a threshold or checking whether a metric meets a target.

Syntax

```
result = value_1 operator value_2
```

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 8	False
<	Less than	5 < 8	True
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	5 <= 3	False

Example

```
marks = 72
pass_mark = 40
print(marks > pass_mark)
print(marks == 72)
print(marks != 100)
print(marks <= pass_mark)
```

Output

```
True
True
True
False
```

Data/AI Example

```
actual_sales = 125000
target_sales = 150000
target_met = actual_sales >= target_sales
print(target_met)
```

Output

```
False
```

Important Points

- Every comparison returns a bool that can be stored in a variable.

Common Mistakes

- Writing = instead of == when comparing.
- Comparing a string with a number. "5" == 5 is False because the types differ.

Interview Notes

- **Q:** What does a comparison operator return?
A: A boolean value, True or False.

2.5.3 Logical Operators

What it is

Logical operators combine or reverse boolean values: and, or and not.

Why it matters

Real rules usually have more than one condition, such as customers within an age range who also have an income above a limit. Logical operators combine those conditions.

Syntax

```
condition_1 and condition_2
condition_1 or condition_2
not condition
```

Example

```
attendance = 82
marks = 68
is_eligible = attendance >= 75 and marks >= 40
print(is_eligible)
print(attendance < 75 or marks < 40)
print(not is_eligible)
```

Output

```
True
False
False
```

Explanation

- and is true only when both conditions are true.
- or is true when at least one condition is true.
- not reverses the result.

Data/AI Example

```
age = 34
income = 52000
is_target_customer = age >= 25 and age <= 45 and income > 40000
print(is_target_customer)
```

Output

```
True
```

Important Points

- Python evaluates and and or from left to right and stops as soon as the result is known.
- Use brackets when mixing and and or to make the meaning clear.

Common Mistakes

- Writing `age >= 25 and <= 45`. Each side of and must be a full condition: `age >= 25 and age <= 45`.

Interview Notes

- **Q:** Which has higher precedence, and or or?
A: and is evaluated before or. Brackets remove any doubt.

A	B	A and B	A or B
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

2.5.4 Assignment Operators

What it is

Assignment operators store a value in a variable. The basic operator is `=`. Compound operators such as `+=` update a variable using its current value.

Why it matters

Updating running values is common: accumulating totals, counting records and adjusting values inside loops.

Syntax

```
variable = value
variable += value
```

Example

```
total_sales = 1000
total_sales += 250
print(total_sales)
total_sales -= 100
print(total_sales)
total_sales *= 2
print(total_sales)
total_sales /= 4
print(total_sales)
```

Output

```
1250
1150
2300
575.0
```

Data/AI Example

```
total_revenue = 0
total_revenue += 15000
total_revenue += 22000
total_revenue += 18500
print(total_revenue)
```

Output

```
55500
```

Important Points

- A compound operator is a shorter way to write a calculation and assignment together.

Common Mistakes

- Using += on a variable that has not been created yet, which raises `NameError`.
- Forgetting that /= turns an integer into a float.

Interview Notes

- **Q:** What is the difference between `x = x + 1` and `x += 1`?
- **A:** They give the same result. `x += 1` is the shorter form.

Operator	Example	Same as
=	x = 5	Assign 5 to x
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
//=	x //= 3	x = x // 3
%=	x %= 3	x = x % 3
**=	x **= 3	x = x ** 3

2.6 Practical Examples

These examples combine variables, arithmetic and comparison operators into small analytics tasks.

Calculation	Formula
Total	total = value_1 + value_2 + value_3

Calculation	Formula
Average	$\text{average} = \text{total} / \text{count}$
Difference	$\text{difference} = \text{new_value} - \text{old_value}$
Growth percentage	$\text{growth} = (\text{new_value} - \text{old_value}) / \text{old_value} * 100$

2.6.1 Student Marks and Average

What it is

Calculate the total and average marks of a student and check whether the student passed.

Example

```

maths = 85
science = 78
english = 92
total_marks = maths + science + english
average_marks = total_marks / 3
print("Total marks:", total_marks)
print("Average marks:", average_marks)
print("Passed:", average_marks >= 40)

```

Output

```

Total marks: 255
Average marks: 85.0
Passed: True

```

Explanation

- The three marks are added to get the total.
- Dividing by the number of subjects gives the average, which is a float.
- The comparison `average_marks >= 40` produces a boolean.

Important Points

- Average = total divided by the number of values.

Common Mistakes

- Dividing by the wrong count, or forgetting to add all values before dividing.

2.6.2 Sales Difference

What it is

Find how much sales changed between two months.

Example

```

sales_last_month = 120000
sales_this_month = 145000
sales_difference = sales_this_month - sales_last_month
print("Sales difference:", sales_difference)
print("Sales increased:", sales_difference > 0)

```

Output

```
Sales difference: 25000
Sales increased: True
```

Explanation

- Subtracting the older value from the newer value gives the change.
- A positive result means an increase and a negative result means a decrease.

Important Points

- Always subtract in the same direction: new value minus old value.

Common Mistakes

- Reversing the subtraction, which flips the sign and gives a misleading result.

2.6.3 Sales Growth Percentage

What it is

Express the change in sales as a percentage of the earlier value.

Why it matters

A difference of 18,000 means different things for sales of 120,000 and 1,200,000. Growth percentage makes changes comparable across products, regions and periods.

Syntax

```
growth_percentage = (new_value - old_value) / old_value * 100
```

Examples

```
sales_last_month = 120000
sales_this_month = 138000
growth_percentage = (sales_this_month - sales_last_month) / sales_last_month * 100
print("Growth percentage:", growth_percentage)
```

Output

```
Growth percentage: 15.0
```

```
sales_last_month = 150000
sales_this_month = 135000
growth_percentage = (sales_this_month - sales_last_month) / sales_last_month * 100
print("Growth percentage:", growth_percentage)
```

Output

```
Growth percentage: -10.0
```

Explanation

- The brackets make Python subtract first, then divide by the old value.
- Multiplying by 100 converts the ratio to a percentage.
- The second example shows that a decrease gives a negative percentage.

Important Points

- The change is always divided by the old value, not the new value.
- A negative result means sales declined.

Common Mistakes

- Dividing by the new value instead of the old value.
- Omitting the brackets, which changes the order of calculation.
- Using an old value of zero, which raises `ZeroDivisionError`.

Interview Notes

- **Q:** How do you calculate percentage growth?
A: (New value - old value) divided by old value, multiplied by 100.

2.7 Section Summary

Concept	Key point
Variables	Names that refer to values. <code>=</code> assigns, <code>==</code> compares.
<code>int</code>	Whole numbers: counts, quantities, epochs.
<code>float</code>	Decimal numbers. Stored approximately, so use <code>round()</code> for display.
<code>str</code>	Text in quotes. <code>+</code> joins strings and <code>*</code> repeats them.
<code>bool</code>	<code>True</code> or <code>False</code>
<code>type()</code>	Returns the type of a value. It does not change it.
Type conversion	<code>int()</code> , <code>float()</code> , <code>str()</code> , <code>bool()</code> . Implicit conversion turns <code>int + float</code> into <code>float</code> .
Arithmetic operators	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>//</code> <code>%</code> <code>**</code> . Brackets control the order.
Comparison operators	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code> return a boolean.
Logical operators	<code>and</code> , <code>or</code> , <code>not</code> combine conditions.
Assignment operators	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>//=</code> <code>%=</code> <code>**=</code> store and update values.